École Hassania des Travaux Publics
Ingénieur d'État – [your track, e.g. Mathematics & Systems Engineering / IT Engineering]

Internship Report — Rapport de Stage

Log Anomaly Detection with Hawkes-Process Timing and LLM Triage on System Logs

An unsupervised, rigorously validated pipeline for security-log analysis, developed within OCP Group's Digital Infrastructure and Operations

Author: El Alami Amine
Host organisation: OCP Group — Digital Infrastructure and Operations
Company supervisor: [Company supervisor's name]
Academic supervisor: [Academic supervisor's name]
Internship period: [Start date] – [End date], 2026

Academic year 2025–2026

Acknowledgements

I would like to express my sincere gratitude to my company supervisor, [Company supervisor's name], for the guidance, trust, and autonomy granted throughout this internship, and to the whole Digital Infrastructure and Operations team at OCP Group for their welcome, their availability, and the many exchanges that anchored an academically oriented project in operational reality.

I warmly thank my academic supervisor, [Academic supervisor's name], for the methodological guidance, and the faculty of the École Hassania des Travaux Publics for the scientific foundations—in probability, statistics, and computer science—on which this work directly rests.

Finally, I thank OCP Group for offering an environment in which an intern is encouraged to pursue technical depth and intellectual honesty, and everyone who took the time to review and challenge the results presented here.

Abstract

This report documents an internship carried out within OCP Group's *Digital Infrastructure and Operations*: the design, implementation, and rigorous validation of an end-to-end anomaly-detection system for machine-generated system logs, augmented with a stochastic-process model of event timing and a large-language-model (LLM) triage layer. The system is developed and evaluated entirely on public benchmark datasets (`loghub` HDFS and BGL), making every result reproducible while remaining directly relevant to the security-operations context of a large industrial group. The pipeline comprises: automatic log parsing into structured events (Drain); an unsupervised autoencoder detector benchmarked against an Isolation Forest; a univariate Hawkes point process fitted by maximum likelihood to model the self-excitation of log events, validated with the time-rescaling theorem and evaluated as a source of detection features; an adaptive alerting threshold based on extreme-value theory, with a distribution-free alternative, stabilising the false-alarm rate under drift; and an LLM triage agent that deduplicates alerts, correlates them into incidents, and produces ranked, structured, explained incident reports. Validation follows a deliberately strict protocol: a never-shuffled temporal train/test split, precision–recall analysis rather than accuracy, false alarms per day at a fixed detection rate as the headline metric, and explicit measurement of drift. Key results include a detection PR-AUC of 0.987 on HDFS ($\sim$20$\times$ fewer false alarms than the baseline), a near-critical Hawkes branching ratio ($\eta \approx 0.97$–1.0) on BGL, a measured halving of false alarms attributable to timing features, and a demonstrated stabilisation of the false-alarm rate under drift. Negative and nuanced findings are reported with the same care as positive ones, as an explicit methodological choice.

Keywords: anomaly detection, system logs, Hawkes process, point processes, extreme-value theory, unsupervised learning, autoencoder, LLM, security operations, OCP Group.

Résumé

Ce rapport présente le travail réalisé lors d'un stage au sein de la direction *Digital Infrastructure and Operations* du Groupe OCP : la conception, l'implémentation et la validation rigoureuse d'une chaîne complète de détection d'anomalies dans les journaux système (*logs*), enrichie d'un modèle stochastique de la dynamique temporelle des événements et d'une couche de tri assistée par grand modèle de langage (LLM). Le système est développé et évalué exclusivement sur des jeux de données publics de référence (`loghub` : HDFS et BGL), ce qui rend chaque résultat reproductible tout en restant directement transposable au contexte opérationnel d'un grand groupe industriel. La chaîne comprend : l'analyse syntaxique automatique des logs (Drain) ; un détecteur non supervisé par auto-encodeur comparé à une forêt d'isolation ; un processus de Hawkes univarié estimé par maximum de vraisemblance pour modéliser l'auto-excitation des événements, validé par le théorème de changement de temps ; un seuillage adaptatif fondé sur la théorie des valeurs extrêmes ; et un agent de tri LLM produisant des incidents hiérarchisés, structurés et expliqués. La validation suit un protocole strict : séparation temporelle jamais mélangée, analyse précision–rappel, fausses alertes par jour à taux de détection fixé comme métrique principale, et mesure explicite de la dérive. Les résultats négatifs ou nuancés sont rapportés avec le même soin que les résultats positifs, par choix méthodologique.

Mots-clés : détection d'anomalies, journaux système, processus de Hawkes, valeurs extrêmes, apprentissage non supervisé, auto-encodeur, LLM, cybersécurité, Groupe OCP.

Contents

List of Figures

List of Tables

Acronyms

AE	Autoencoder
API	Application Programming Interface
BGL	Blue Gene/L (supercomputer log dataset)
EDA	Exploratory Data Analysis
EHTP	École Hassania des Travaux Publics
EVT	Extreme-Value Theory
FA	False Alarm
GPD	Generalized Pareto Distribution
HDFS	Hadoop Distributed File System (log dataset)
IT / OT	Information Technology / Operational Technology
KS	Kolmogorov–Smirnov (test)
LLM	Large Language Model
MLE	Maximum-Likelihood Estimation
MLP	Multilayer Perceptron
MSE	Mean Squared Error
OCP	OCP Group (historically, Office Chérifien des Phosphates)
POT	Peaks-Over-Threshold
PR-AUC	Area Under the Precision–Recall Curve
RAS	Reliability, Availability, Serviceability
SOC	Security Operations Centre
UM6P	Mohammed VI Polytechnic University

Part I

Host organisation and internship context

Chapter 1

General introduction

1.1 Context

Modern industrial organisations run on software. Behind every mine conveyor, every chemical unit, every logistics chain and every commercial transaction of a group such as OCP stands a digital estate of servers, networks, applications, and industrial control systems—and every element of that estate continuously writes *logs*: timestamped text records of what it is doing. Logs are simultaneously the primary forensic record of an infrastructure and an overwhelming torrent of data: a large organisation produces millions of lines per day, in which the events that actually matter —failures, misconfigurations, intrusions, cascading faults—are rare, unlabelled, and buried.

This internship, hosted by OCP Group's *Digital Infrastructure and Operations*, addresses that problem end to end. It designs, implements, validates, and documents a complete prototype that ingests raw logs and produces a short, ranked, explained list of incidents. Beyond the engineering, the internship had an explicit academic ambition, agreed with both supervisors: to treat the subject as a case study in *rigorous time-series methodology*—unsupervised learning under extreme class imbalance, point-process modelling of event timing, extreme-value methods for alerting, and a validation discipline strict enough to be defensible in front of a critical audience.

1.2 Problem in one paragraph

Given a stream of raw log lines, with no labelled examples of "bad" behaviour, the system must: structure the stream into events; learn what normal activity looks like; score departures from normality; convert scores into alerts whose false-alarm rate an operations team can live with, even as the data drifts; and compress the surviving alerts into a handful of prioritised, explained incidents. Each stage of that sentence maps to a chapter of this report.

1.3 Objectives of the internship

The objectives fixed at the outset were:

1. build a complete, reproducible pipeline: raw logs → structured events → unsupervised detection → calibrated alerting → triage → demonstrator;
2. place a *stochastic-process model* at the mathematical core: fit a Hawkes process to event timestamps, estimate its branching ratio, validate the fit with the time-rescaling theorem, and evaluate the resulting timing features for detection;
3. apply a strict validation protocol throughout: temporal train/test split, precision–recall analysis, false alarms per day as headline metric, drift measurement;
4. prototype an LLM-based triage agent producing structured, ranked, explained incidents;
5. (stretch) investigate extreme-value theory for self-calibrating alert thresholds.

All five objectives, including the stretch goal, were completed.

1.4 Reading guide

The report is organised in three parts. **Part I** (Chapters 2–3) presents the host organisation—OCP Group, then the Digital Infrastructure and Operations entity—and the organisation of the internship itself. **Part II** (Chapters 4–5) defines the problem formally, reviews the state of the art while explicitly separating prior work from this project's contribution, and develops the theoretical background from first principles. **Part III** (Chapters 6–9) presents the methodology, the complete implementation and results—including the measurements that redirected the design—the discussion of limitations, and the conclusion. Appendices provide notation, a glossary, reproduction instructions, and a description of the automated test suite.

Chapter 2

Presentation of OCP Group

2.1 Identity and mission

OCP Group (historically *Office Chérifien des Phosphates*) is a Moroccan public limited company and one of the world's leading producers and exporters of phosphate rock, phosphoric acid, and phosphate-based fertilizers. Its mission sits at the crossroads of industry and food security: phosphorus, extracted from phosphate rock, is one of the three essential macronutrients of agriculture and cannot be substituted. Morocco holds the large majority of the world's known phosphate reserves; OCP, as their operator, therefore occupies a systemic position in the global fertilizer supply chain and a central position in the Moroccan economy, as one of the country's largest employers, investors, and export earners. The group serves customers on five continents, with a particular strategic emphasis on the development of African agriculture.

2.2 A century of history

Table 2.1: Selected milestones of OCP Group.

Year	Milestone
1920	Creation of the Office Chérifien des Phosphates; mining begins at Khouribga the following year.
1930s–1960s	Expansion of mining to the Gantour basin (Youssoufia, then Benguerir); first industrial beneficiation capacities.
1965	Start of chemical valorisation at Safi: the group moves downstream from rock to phosphoric acid and fertilizers.
1975–1986	Development of the Boucraa mine and the Jorf Lasfar chemical platform, which becomes the group's flagship processing hub.
2008	Transformation into a public limited company, *OCP S.A.*—the starting point of a decade of industrial modernisation.
2014	Commissioning of the Khouribga–Jorf Lasfar slurry pipeline ($\approx$187 km), a structural change in the group's logistics.
2017	Foundation of Mohammed VI Polytechnic University (UM6P) in Benguerir; creation of digital joint ventures with global technology partners.
2020s	Green-investment programme: desalination, renewable energy, green ammonia; continuation of the group-wide digital transformation.

2.3 The integrated value chain

OCP is vertically integrated from the mine to the port:

1. **Extraction.** Open-pit mining in the Khouribga and Gantour basins and at Boucraa; the ore is screened and enriched on site (washing, flotation, drying).
2. **Transport.** The slurry pipeline carries phosphate pulp from Khouribga to the coast, replacing rail transport, cutting logistics costs substantially and reducing the water and energy intensity of each transported tonne.
3. **Chemical processing.** The Jorf Lasfar platform—among the world's largest fertilizer complexes—and the historic Safi platform transform rock into phosphoric acid and finished fertilizers (DAP, MAP, TSP, NPK blends).
4. **Commercialisation.** Products are shipped worldwide from the group's port facilities, with tailored formulations for different soils and crops—notably through soil-mapping programmes in Africa.

2.4 Strategy: sustainability and knowledge

Two strategic axes frame the group's current decade. The first is **sustainability**: flagship programmes aim at covering industrial water needs from non-conventional sources (desalination and treated wastewater), powering operations with renewable electricity, and developing green-ammonia value chains for low-carbon fertilizers. The second is **knowledge and talent**: with the foundation of UM6P, an international-standard research university, and an ecosystem of programmes linking research, entrepreneurship, and industry, the group has positioned itself as an anchor of Morocco's scientific and digital skills ecosystem—the very ecosystem in which engineering-school internships such as this one take place.

2.5 Products, markets, and key figures

The group's portfolio spans the full chain of phosphate valorisation: phosphate rock itself; phosphoric acid (merchant grade and purified); and finished fertilizers—di-ammonium and mono-ammonium phosphate (DAP/MAP), triple superphosphate (TSP), and an expanding range of NPK blends and speciality products adapted to specific soils and crops. Commercially, the group serves all major agricultural regions; its African strategy couples supply with agronomic support (soil maps, farmer programmes), consistent with its food-security mission.

Table 2.2: Order-of-magnitude key figures (public reporting; values vary by year and are given as orders of magnitude).

Founded	1920 (became OCP S.A. in 2008)
Headquarters	Casablanca, Morocco
Workforce	∼20,000 employees
Position	Among the world's leading phosphate producers and exporters
Reserves basis	Morocco holds the large majority of world phosphate reserves
Main mining sites	Khouribga, Gantour (Benguerir, Youssoufia), Boucraa
Processing hubs	Jorf Lasfar (flagship), Safi
Logistics flagship	Khouribga–Jorf Lasfar slurry pipeline (≈187 km)
Research & education	UM6P (founded 2017), group R&D programmes

2.6 Corporate social engagement

Beyond industry, the group runs structured social programmes—community development around its sites, support for education and entrepreneurship, and employee civic-engagement initiatives—reflecting its position as a national institution as much as a company. These programmes matter

to this report only as context: they illustrate an organisational culture in which an intern's contribution is expected to meet professional standards of rigour and documentation, the standard this report attempts to honour.

2.7 The digital ecosystem

The group's digital transformation is supported by dedicated structures and partnerships, publicly visible among which are: joint ventures with global technology firms for IT services and infrastructure modernisation (notably *Teal Technology Services*, created with IBM); the engineering joint venture *JESA*; and the research and innovation environment of UM6P with its associated entities. Digital programmes span the operational spectrum—mine digitalisation, industrial performance, predictive maintenance, integrated supply-chain management—and the corporate spectrum — ERP, data platforms, collaboration. This transformation is the direct context of the host entity presented in the next chapter: a larger, more connected digital estate multiplies both the value and the attack surface of the group's information systems.

Chapter 3

Host entity and internship organisation

3.1 Digital Infrastructure and Operations

The internship was hosted by **Digital Infrastructure and Operations**, the entity of OCP Group responsible for building and running the digital foundation on which the group's applications and industrial digital services operate. Its remit spans, in general terms: data centres and cloud infrastructure; enterprise networks connecting mining sites, chemical platforms, ports, and offices; the operation and monitoring of platforms and services (availability, performance, capacity); and the operational side of cybersecurity—monitoring, detection, and response, in coordination with the group's security governance.

Confidentiality

This report deliberately describes the host entity at the level of its public mission. No internal architecture, tooling, dataset, or procedure of OCP Group is described or used anywhere in this work; the project was executed entirely on public benchmark data precisely so that it could be documented openly and reproduced by anyone.

3.2 Why log analytics matters to this entity

An infrastructure-and-operations organisation lives in its telemetry. Every layer it operates—compute, network, storage, platforms, security appliances—emits logs continuously, and three structural difficulties define the analytical problem, none specific to OCP but all amplified by industrial scale:

1. **Volume asymmetry.** Events worth human attention are a vanishing fraction ($\sim 10^{-3}$ or less) of millions of daily lines; manual review is impossible and naive alerting drowns analysts in false positives—the well-documented "alert fatigue" of security operations.
2. **Absence of labels.** New failure modes and attack techniques appear continuously; no labelled catalogue can be exhaustive. Methods must learn *normality* and flag departures—the unsupervised paradigm.
3. **Temporal clustering.** Real incidents are *cascades*: one root cause triggers bursts of correlated events across systems. Timing carries information, yet most deployed tooling treats events as independent points.

These three difficulties are, term for term, the three axes of this project: unsupervised detection (§7.3), explicit modelling of temporal clustering (§7.4), and volume reduction by correlation and triage (§7.7).

3.3 Anatomy of a security-operations workflow

To situate the project precisely, consider the canonical workflow of a security-operations capability, common to all large organisations:

1. **Collection**: logs and telemetry are centralised from servers, network equipment, applications, and security appliances;
2. **Normalisation**: heterogeneous formats are parsed into structured events;
3. **Detection**: rules and models score events or aggregates for suspicion;
4. **Alerting**: scores exceeding thresholds become alerts;
5. **Triage**: analysts deduplicate, group, prioritise, and investigate alerts;
6. **Response and feedback**: incidents are remediated and lessons fed back into detection content.

The pain points concentrate in stages 3–5: rule-based detection misses novel behaviour (hence unsupervised models); static thresholds break under drift (hence adaptive alerting); and per-alert handling drowns analysts (hence automated correlation and triage). The project's components are a one-to-one mapping onto these three pain points—stages 2 through 5 of this workflow are precisely what the prototype implements end to end on public data.

3.4 Missions of the internship

The intern's mission was to design, implement, validate, and document the complete prototype, working autonomously with regular supervision checkpoints. Concretely, the missions were: (i) establish the data and parsing foundation on public benchmarks; (ii) build and validate the detection layer; (iii) develop the Hawkes timing model and its diagnostics; (iv) design drift-robust alerting; (v) prototype the triage agent and a demonstrator; and (vi) document everything to an academic standard, including the present report and a fully reproducible repository.

3.5 Work organisation and planning

The work was organised in weekly increments, each closed by a committed, tested milestone. Table 3.1 summarises the effective schedule.

Table 3.1: Internship planning (effective).

Weeks	Content
1	Project scoping; repository and tooling setup; data acquisition; Drain parsing of the HDFS sample and full dataset; first data-quality checks.
2	Exploratory data analysis; discovery of the timestamp-resolution problem; decision to host the timing model on BGL; feature design and temporal split.
3	Detection layer: Isolation Forest baseline, autoencoder, evaluation harness (PR analysis, false alarms/day, drift decomposition).
4	Hawkes core: likelihood, recursion, simulation, estimator verification; BGL parsing and EDA.
5	Hawkes fits on contrasting BGL windows; time-rescaling diagnostics; timing-feature ablation.
6	Adaptive thresholding: EVT implementation and verification; discovery of the discrete-score failure mode; rolling-quantile alternative; drift demonstration.
7	Triage agent (deduplication, correlation, explanation backends); Streamlit demonstrator; end-to-end integration.
8	Consolidation: test suite completion, documentation, analysis notebook, and the present report.

3.6 Tools and working environment

Table 3.2: Technical environment.

Domain	Tools
Language & core	Python 3.11; NumPy, pandas, SciPy
Machine learning	scikit-learn (Isolation Forest, metrics), PyTorch (autoencoder)
Log parsing	`logparser` (Drain), loghub benchmark configurations
Point processes	own implementation (likelihood, recursion, Ogata thinning, time-rescaling) in SciPy
Visualisation	Matplotlib; Streamlit for the demonstrator
Engineering	git; pytest (34 automated tests); reproducible scripts; Jupyter notebook for analysis
Documentation	LaTeX (this report), Markdown technical notes

Part II

Problem, state of the art, and theoretical background

Chapter 4

Problem statement and state of the art

4.1 Formal problem statement

Let a log source emit raw text lines. After parsing, the stream becomes a marked point pattern $\{(t_i, e_i)\}_{i \geq 1}$: $t_i \in \mathbb{R}_+$ is a timestamp and $e_i \in \{1, \ldots, K\}$ an *event type* from a finite vocabulary of K templates. Events are grouped into *units of analysis* u (sessions or fixed windows), each with a latent state $y_u \in \{0, 1\}$ (normal / anomalous). The task decomposes into four sub-problems:

1. **Scoring** (unsupervised): learn $s(u) \in \mathbb{R}$ from *unlabelled* data such that anomalous units rank systematically higher; labels serve only for evaluation.
2. **Timing modelling**: model the law of the timestamps $\{t_i\}$ themselves, both as a scientific object (how do events beget events?) and as a feature source for scoring.
3. **Alerting**: map scores to binary alerts through a threshold whose false-alarm behaviour is controlled *through time*, i.e. robust to drift of the score distribution.
4. **Triage**: compress the alert stream into few, ranked, explained incidents.

Three empirical properties constrain all four: extreme class imbalance ($\Pr[y_u{=}1] \sim 10^{-3}$–$10^{-1}$); non-stationarity at the scale of hours; and strong temporal clustering of events. The validation protocol (§6.5) is built around exactly these properties.

4.2 State of the art

4.2.1 Log parsing

Parsing free text into a finite template vocabulary is a mature problem with public benchmarks. **Drain** [1] maintains a fixed-depth parse tree whose internal nodes route lines by length and leading tokens and whose leaves hold template clusters; each line is assigned online, in near-constant time, to the most similar cluster or to a new one. Drain is the accuracy/speed reference of the `logparser` toolkit, evaluated across sixteen datasets in `loghub` [2], and ships with benchmark configurations per dataset—which this project adopts unchanged for comparability.

4.2.2 Log anomaly detection

Two families dominate. *Count-based* methods aggregate each session/window into an event-count vector and apply an unsupervised detector: PCA reconstruction, invariant mining, clustering, or Isolation Forest [6]; the `loglizer` study [3] systematises them. *Sequence-based* deep methods model order: DeepLog [4] trains an LSTM to predict the next template and flags low-probability continuations; LogAnomaly [5] adds template semantics; autoencoders on count vectors are the standard unsupervised neural baseline. On the HDFS benchmark these methods routinely reach F1/PR-AUC of 0.95–0.99; the result is among the most reproduced in the field—an important fact for positioning (§4.3).

4.2.3 Point processes for event streams

The Hawkes process [7] is the canonical self-exciting point process, with mature theory: likelihood-based estimation, the cluster/branching representation, exact simulation by thinning [8], and goodness of fit via the time-rescaling theorem [9]; textbook treatments include Daley and Vere-Jones [10]. It is standard in seismology (aftershock modelling), financial microstructure (order-flow), and social-media diffusion. Its use as a *feature source for log anomaly detection* is uncommon: the dominant log-analysis paradigm discards fine-grained timing altogether, which is precisely the gap this project probes.

4.2.4 Thresholding and extreme values

Converting scores to alerts is usually treated as an afterthought (fixed quantiles), yet it is where deployments fail under drift. Extreme-value theory provides the principled tool: the Pickands–Balkema–de Haan theorem [11] (textbook: Coles [12]) characterises threshold exceedances by the Generalized Pareto law, and the SPOT/DSPOT algorithms [13] apply it to streaming anomaly scores with re-estimation over time.

4.2.5 LLM-assisted triage

Employing large language models to summarise, categorise, and prioritise security alerts is an emerging, practice-driven area. Two design lessons recur in early literature and industrial reports: constrain the model to a *structured output schema* (free text is not actionable downstream), and apply LLMs *after* deterministic reduction—deduplication and correlation—so that cost scales with incidents, not raw alerts. Both lessons are adopted here.

4.3 Positioning: prior art versus contribution

Table 4.1 makes the boundary explicit, component by component. In particular, the strong HDFS detection figure obtained in Chapter 7 *reproduces* a well-known benchmark outcome; it is claimed only as evidence that the pipeline is correctly built. The centre of gravity of the contribution lies in the timing model and its honest evaluation, the validation protocol, the drift-aware thresholding, and the correlation-grounded triage.

Table 4.1: Prior art versus this project's contribution.

Component	**Established prior work**	**Contribution of this project**
Parsing	Drain, benchmark configurations [1, 2].	Used unchanged, for comparability.
Detection	Count-vector AE / IsoForest on HDFS widely reproduced [3, 4].	Reproduced as a validated baseline; added value: the strict protocol and drift analysis around it.
Timing	Hawkes theory and diagnostics mature elsewhere [7, 9, 10].	Application to logs: branching-ratio estimation on BGL, rescaling validation, measured feature ablation.
Thresholding	EVT/POT theory [11, 12]; SPOT [13].	Implementation, verification, documented failure mode on discrete scores, drift-stabilisation demonstration.
Triage	LLM alert summarisation (emerging).	Correlation grounded in self-excitation; strict JSON schema; dedup-before-LLM design.
Validation	Temporal splits, PR analysis known in principle.	Applied end-to-end without exception; FA/day headline; per-period drift reporting.

Chapter 5

Theoretical background

This chapter develops, from first principles, every mathematical tool used in Part III. A reader comfortable with probability at engineering-school level should be able to follow all of Part III from this chapter alone.

5.1 Evaluation under extreme class imbalance

5.1.1 Why accuracy fails

With P positives and N negatives, $P \ll N$, the degenerate classifier "always normal" attains accuracy $N/(N+P)$—above 97% on our data —while detecting nothing. Any metric dominated by true negatives is uninformative here; evaluation must focus on the rare class.

5.1.2 Precision, recall, and the PR curve

For a threshold θ on the score, with TP, FP, FN the usual counts:

$$\text{precision}(\theta) = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall}(\theta) = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Sweeping θ traces the PR curve; its area (**PR-AUC**, equivalently average precision) summarises ranking quality. Two facts make it the right choice under imbalance. First, its *chance level equals the prevalence*: a scorer independent of the labels has, at any recall, expected precision $P/(P+N)$, so the whole random curve is the horizontal line at the positive rate, and any lift above it is interpretable. Second, it ignores true negatives entirely, so the huge N cannot flatter it. ROC-AUC, in contrast, normalises the false-positive count by N; with $N \sim 10^5$, tens of thousands of false alarms move the ROC curve imperceptibly while destroying practical utility—which is why ROC figures are not reported in this work.

5.1.3 The operational metric

Operations teams commit to a detection level and live with the alert volume. The headline metric is therefore **false alarms per day at fixed recall**: fix recall at 0.90 (the smallest threshold whose top-scored set captures 90% of positives), count false positives above it, and divide by the *measured* duration of the test period. Reporting the duration honestly matters: test units are rarely uniform in time, and assuming a nominal period silently rescales the metric.

5.2 Temporal validation and leakage

The purpose of a test set is to estimate performance on *future* data. Random shuffling interleaves training and test in time, allowing information from the test period (shared regimes, shared

incidents, long-lived sessions) to reach the model—**data leakage**—and inflating every metric. The protocol here is a strict **temporal split**: order units by start time, train on the earliest fraction, test on the strictly later remainder, and never shuffle. The prohibition covers *everything* fitted on data: normalisation statistics, feature vocabularies, and alert thresholds are all computed on the training period only. The cost is honesty: temporal evaluation yields lower, real numbers, and exposes drift instead of averaging it away.

5.3 Log parsing: the Drain algorithm

Because every later stage consumes Drain's output, its mechanism deserves a precise description. Drain [1] maintains a parse tree of fixed depth. An incoming line is first preprocessed by *masking*: known variable patterns (block identifiers, IP addresses, numbers) are replaced by a wildcard, using dataset-specific regular expressions. The masked line is then routed: the first tree level branches on the token *count* of the line; subsequent levels branch on the first few tokens; the leaf reached holds a list of *template clusters*. The line is compared to each cluster's template by token-wise similarity $\text{sim} = \frac{1}{L}\sum_{k=1}^{L} \mathbf{1}\{w_k = \tilde{w}_k\}$ (wildcards match anything); if the best similarity exceeds a threshold (0.5 in the benchmark configuration) the line joins that cluster and mismatching tokens become wildcards, otherwise a new cluster is created. *Worked example*: the masked lines "`Received block <*> of size <*> from <*>`" and "`Received block <*> of size <*> from <*>`" route to the same leaf (same length, same head tokens) and merge at similarity 1; whereas "`Deleting block <*> file <*>`" differs in length and lands in a different branch, founding its own template. The result is an online, near-constant-time parser whose output vocabulary is stable and human-readable—properties that machine-learned parsers do not always offer, and the reason Drain remains the reference.

5.4 Unsupervised detectors

5.4.1 Why the chance level of PR-AUC equals the prevalence

A short derivation used repeatedly when reading results. If scores are independent of labels, then for any threshold the flagged set is a uniformly random subset of the units; the expected fraction of true positives inside it equals the population prevalence $\pi = P/(P{+}N)$. Hence expected precision is π *at every recall*, the random PR curve is the horizontal line at height π, and its area is π. This is why we always quote PR-AUC *together with* the prevalence (e.g. "0.987 against a chance level of 0.022")—the ratio, not the absolute value, carries the meaning.

5.4.2 Isolation Forest

Isolation Forest [6] scores by *ease of isolation*. An isolation tree partitions the data by recursive random splits (random feature, random cut point); a point's *path length* is the number of splits until it sits alone. Points in dense regions require many splits; outliers few. Scores aggregate path lengths over an ensemble of trees built on subsamples, normalised by the expected path length of an unsuccessful binary-search-tree lookup, $c(n) = 2H_{n-1} - 2(n{-}1)/n$ with H_m the harmonic number, so that scores are comparable across sample sizes. Strengths: no labels, near-linear cost, no distributional assumptions—the natural baseline. Limits: axis-aligned splits capture inter-feature structure only coarsely, which is exactly where a neural detector can improve.

5.4.3 Autoencoders

An **autoencoder** composes an encoder $f_\theta : \mathbb{R}^d \to \mathbb{R}^k$ and decoder $g_\phi : \mathbb{R}^k \to \mathbb{R}^d$, $k \ll d$, trained to minimise reconstruction error $\|x - g_\phi(f_\theta(x))\|^2$. The bottleneck forbids the identity; minimising the loss forces the network to encode the dominant regularities of the training distribution. Trained

exclusively on normal units, it reconstructs normal patterns accurately and abnormal patterns poorly; the per-unit MSE is the anomaly score. Practical safeguards used in this work: inputs are $\log(1+x)$-transformed (heavy-tailed counts) and standardised with statistics from *normal training data only*; training uses Adam with early stopping on a held-out slice of the normal training set (never on anything from the test period); and the score orientation (higher = more anomalous) matches the baseline so a single evaluation harness serves both.

5.5 Point processes and the Hawkes model

5.5.1 Counting processes and conditional intensity

A temporal point process is a random sequence of times $0 < t_1 < t_2 < \cdots$, equivalently a counting process $N(t) = \#\{i : t_i \le t\}$. Its dynamics are specified by the **conditional intensity**

$$\lambda(t) = \lim_{\Delta \downarrow 0} \frac{\Pr[N(t+\Delta) - N(t) = 1 \mid \mathcal{H}_t]}{\Delta},$$

the instantaneous event rate given the history $\mathcal{H}_t$. The homogeneous Poisson process is the memoryless case $\lambda \equiv \mu$: exponential i.i.d. gaps, independent counts, Fano factor (variance/mean of interval counts) equal to 1. Measured Fano factors of 5–2000 on our data (§7.2) rule it out and call for history dependence.

5.5.2 The Hawkes process and its branching structure

The exponential-kernel **Hawkes process** [7] sets

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t - t_i)}, \qquad \mu, \alpha, \beta > 0. \tag{5.1}$$

Each event lifts the intensity by α; the lift decays with timescale $1/\beta$. The process has an exact *cluster representation*: immigrants arrive as a Poisson process of rate μ, and every event (immigrant or descendant) independently produces offspring as an inhomogeneous Poisson process with rate $\alpha e^{-\beta s}$ after it. The expected number of direct offspring per event, the **branching ratio**, is

$$\eta = \int_0^\infty \alpha e^{-\beta s}\, ds = \frac{\alpha}{\beta}. \tag{5.2}$$

Galton–Watson theory then gives the phase diagram: for $\eta < 1$ (subcritical) cascades are a.s. finite with mean total size $1/(1-\eta)$ per immigrant, and the process is stationary with mean rate

$$\bar{\lambda} = \frac{\mu}{1-\eta} \quad \text{(immigrants } \mu\text{, each spawning a cascade of mean size } \tfrac{1}{1-\eta}\text{)},$$

whereas $\eta \ge 1$ is (super)critical: cascades become self-sustaining and the event count explodes. Thus η condenses "how cascade-prone is this stream?" into a single interpretable number, and its proximity to 1 measures how close the system is to self-sustaining avalanches.

5.5.3 Likelihood, recursion, and estimation

For events $\{t_i\}_{i=1}^n$ on $[0, T]$ the log-likelihood of any intensity model is

$$\ell = \sum_{i=1}^{n} \log \lambda(t_i) - \int_0^T \lambda(s)\, ds, \tag{5.3}$$

the first term rewarding intensity placed on actual events, the second (the expected event count) penalising indiscriminate intensity. For the exponential kernel the integral is closed-form, $\int_0^T \lambda = \mu T + \frac{\alpha}{\beta}\sum_i (1 - e^{-\beta(T-t_i)})$, and the log terms use Ogata's recursion: with $A_i = \sum_{j<i} e^{-\beta(t_i - t_j)}$,

$$A_i = e^{-\beta(t_i - t_{i-1})}\,(1 + A_{i-1}), \qquad A_1 = 0, \qquad \lambda(t_i) = \mu + \alpha A_i. \tag{5.4}$$

Worked micro-example. Take $\beta = 1$ and events at $t = (0, 0.5, 2)$. Then $A_1 = 0$; $A_2 = e^{-0.5}(1 + 0) = 0.607$; $A_3 = e^{-1.5}(1 + 0.607) = 0.359$: the recursion carries the total decayed excitation forward in one multiplication per event, so each likelihood evaluation costs $O(n)$ instead of the naive $O(n^2)$—the property that makes MLE feasible on 10^5–10^6 events. Estimation maximises (5.3) over $(\log\mu, \log\alpha, \log\beta)$—the log parametrisation makes positivity implicit—with the L-BFGS-B quasi-Newton method.

5.5.4 Goodness of fit: the time-rescaling theorem

Let $\Lambda(t) = \int_0^t \lambda(s)\,ds$ be the **compensator**. The **time-rescaling theorem** [9] states that if $\{t_i\}$ is generated by intensity λ, the rescaled times $\tau_i = \Lambda(t_i)$ form a unit-rate Poisson process; equivalently the increments $\Delta\tau_i = \Lambda(t_i) - \Lambda(t_{i-1})$ are i.i.d. Exp(1). Model checking thus reduces to a distributional test of the $\Delta\tau_i$ against Exp(1): a quantile–quantile plot (points on the diagonal = good fit; the *location* of departures diagnoses *where* the model errs) and a Kolmogorov–Smirnov statistic. Two practical rules applied in this work: the sample mean of the $\Delta\tau_i$ equals 1 under any correctly normalised compensator, providing an independent implementation check; and at $n \sim 10^5$ the KS test rejects every parametric model (its power grows without bound), so the QQ geometry and the KS *magnitude*—not the p-value—carry the meaning.

5.5.5 Simulation: Ogata thinning

Validating an estimator requires data with known parameters. Ogata's thinning [8] simulates exactly: from time t with current bound $M \geq \lambda$, draw a candidate gap $\sim \text{Exp}(M)$, accept the candidate with probability $\lambda(\text{candidate})/M$, update the intensity on acceptance, repeat. Estimator verification by simulate-then-refit is a *gate* in this project: no fit on real data is reported unless the estimator provably recovers known parameters (§7.4).

```
def simulate(mu, alpha, beta, T, seed=None):
    rng = np.random.default_rng(seed)
    events, t, excite = [], 0.0, 0.0      # excite = sum of decayed jumps
    while True:
        M = mu + excite                    # intensity upper bound at t+
        t_new = t + rng.exponential(1.0 / M)
        if t_new > T: break
        excite *= np.exp(-beta * (t_new - t))   # decay to candidate
        if rng.uniform() <= (mu + excite) / M:  # accept w.p. lambda/M
            events.append(t_new)
            excite += alpha                     # the new event's jump
        t = t_new
    return np.asarray(events)
```

Listing 5.1: Ogata thinning for the exponential-kernel Hawkes process (as implemented and tested).

5.6 Extreme-value theory for alert thresholds

The alerting problem: choose a threshold z_q exceeded by a target fraction q of scores, and keep it valid as the score distribution drifts. EVT provides the tail model. By the Pickands–Balkema–de Haan theorem [11, 12], for a broad class of distributions the exceedances over a high threshold

t converge in law to the **Generalized Pareto Distribution**

$$\Pr[X - t > x \mid X > t] \;\approx\; \Big(1 + \frac{\gamma x}{\sigma}\Big)^{-1/\gamma},$$

with shape γ and scale σ. Writing n for the sample size and N_t for the number of exceedances, the unconditional tail is $\Pr[X > z] \approx \frac{N_t}{n}\big(1+\gamma(z-t)/\sigma\big)^{-1/\gamma}$; setting it equal to q and solving for z gives the **POT threshold**

$$z_q = t + \frac{\sigma}{\gamma}\Big[\Big(\frac{qn}{N_t}\Big)^{-\gamma} - 1\Big]. \tag{5.5}$$

For streaming use, (t, γ, σ) are re-estimated on a trailing window while q stays fixed—the SPOT construction [13]. A fast estimator suited to constant-time updates is the method of moments: with m and v the mean and variance of the exceedances,

$$\hat\gamma = \tfrac{1}{2}\Big(1 - \frac{m^2}{v}\Big), \qquad \hat\sigma = m\,(1-\hat\gamma).$$

The theorem's continuity hypothesis is not decorative: on score distributions with large atoms (heavy ties) the GPD approximation of the tail degrades and (5.5) can extrapolate absurdly—a failure mode met, diagnosed, and handled in §7.6, where a *distribution-free* alternative (the empirical $(1-q)$-quantile of a trailing window) is substituted.

Part III

Contribution: methodology, implementation, and results

Chapter 6

Methodology and system design

6.1 Datasets and their roles

Two public datasets from `loghub` [2] were selected for complementary properties (Table 6.1); the decisive property—timestamp resolution—was itself established by measurement (§7.2) rather than assumed.

Table 6.1: Datasets and roles.

	HDFS	**BGL**
Origin	Hadoop Distributed File System	Blue Gene/L supercomputer (RAS log)
Raw volume	11,175,629 lines	4,713,493 lines
Time span	≈ 1.6 days	≈ 214.7 days
Labels	per *block* (2.93% anomalous)	per *line* (7.39% alerts)
Timestamps	1-second resolution	microsecond resolution
Role	detection & triage	Hawkes timing model

In HDFS, files are stored as replicated *blocks*; each block's short lifecycle (allocation, replication, serving, deletion) generates a session of log lines, and ground truth labels each block. In BGL, a reliability log of the Blue Gene/L supercomputer, each line carries an inline alert tag. The datasets therefore also differ in *label granularity* (session vs. line), which dictates different units of analysis.

6.2 Architecture

raw logs → **parsing** → **features** → **detection** → **thresholding** → **triage** → demonstrator
↘ **Hawkes timing model (BGL)** ↗

The implementation is an installable Python package (`src/logtriage/`) with one module per stage—`parsing`, `features`, `models`, `hawkes`, `eval`, `triage`—and one thin script per stage under `scripts/`. Each stage writes artefacts (CSV/NumPy/JSON) consumed by the next (Table 6.2); scripts, tests, the analysis notebook, and the demonstrator all import the same package code, so there is a single source of truth for every computation.

Table 6.2: Artefact flow between stages.

Stage	Consumes	Produces
Parsing	raw `.log`	`*_parsed.csv` (timestamp, event_id, template, ...)
Features (HDFS)	parsed CSV, labels	count matrix (`.npy`), session metadata + split
Detection	count matrix, metadata	test scores (`.npy`), metrics (`.json`)
Hawkes (BGL)	parsed CSV	fitted (μ, α, β), diagnostics, figures
Thresholding	test scores	alert streams, stability figures
Triage	scores, parsed CSV	`incidents.json` (ranked, structured)

6.3 Data schemas

For precision, Table 6.3 documents the canonical parsed schemas that all later stages consume.

Table 6.3: Canonical parsed-event schemas.

Dataset	Column	Content
HDFS	`timestamp`	event time, second resolution
	`event_id` / `template`	Drain template id and string
	`level`, `component`	log level and emitting component
	`block_ids`	block identifiers referenced by the line (session keys)
BGL	`timestamp`	event time, microsecond resolution
	`event_id` / `template`	Drain template id and string
	`label`	inline ground truth (0 normal / 1 alert; evaluation only)
	`node`, `component`, `level`	emitting node and severity fields

6.4 Feature construction

HDFS: block sessions. The unit is the block session (labels are per block). Each session is the count vector $x_u \in \mathbb{N}^{48}$ of its template occurrences, transformed by $\log(1+x)$; for the autoencoder, standardisation uses moments of the *normal training* population only.

BGL: fixed windows, two feature blocks. Lacking a session key, the stream is cut into 60-second windows. The *content* block counts the top-50 templates (vocabulary fixed on the training period). The *timing* block summarises the Hawkes excitation level $A(t) = \sum_{t_i<t} e^{-\beta(t-t_i)}$ along the window (mean and maximum) at two decay scales ($1/\beta = 1\,\text{s}$ and $30\,\text{s}$), plus the event count; it is computed from event *times only*—labels never enter—so it is a legitimate feature under the leakage rules. A window is labelled anomalous (for evaluation only) if it contains at least one alert line.

6.5 Validation protocol

Fixed before any model was trained:

1. **Temporal split**: units ordered by start time; earliest 70% train, latest 30% test; never shuffled. On HDFS: boundary at 2008-11-11 06:00:46; 402,543 training vs. 172,518 test blocks.

2. **Unsupervised training**: the autoencoder sees only the 389,428 *normal training* blocks; scalers and vocabularies are fitted on training data only.
3. **Metrics**: PR-AUC; precision and false alarms/day at recall 0.90; durations measured, not assumed.
4. **Drift**: all metrics recomputed on consecutive chronological slices of the test period, with thresholds fixed once.
5. **Single harness**: every detector emits "higher = more anomalous" scores evaluated by the same code.

6.6 Software engineering and quality assurance

Development followed an incremental, test-gated workflow under git. The automated suite (34 tests) covers the metric implementations (operating points and FA/day on synthetic cases with known answers), the Hawkes machinery (parameter recovery from simulation; rescaling accepting the true model and rejecting a wrong one; compensator monotonicity), the EVT threshold (exceedance-rate control, drift tracking, and the discrete-score behaviour), feature builders (session explosion, window labelling), and triage (deduplication, correlation, ranking). Random seeds are fixed; every figure and number in this report regenerates from the repository scripts (Appendix C). The governing principle: *no component touches real data before passing synthetic tests with known ground truth.*

6.7 Risk management

The main project risks were identified at the outset and mitigated explicitly (Table 6.4); two of them materialised and were absorbed by the planned mitigations.

Table 6.4: Risk register (a posteriori status).

Risk	**Mitigation**	**Outcome**
Data unsuitable for the timing model	Measure timestamp structure in EDA *before* modelling; keep a second dataset in scope	Materialised (HDFS ties); absorbed by relocating Hawkes to BGL
Estimator bugs corrupting results	Simulation-based verification gates; automated tests	No fit reported without passing gates
Tool hypotheses violated (EVT)	Verify on simulation; diagnose on real data; keep a distribution-free fallback	Materialised (discrete scores); absorbed by rolling quantile
Library unavailability	Own implementations of the mathematical core	`tick` unavailable on modern Python; core re-implemented and tested
Scope creep in eight weeks	Weekly committed milestones; stretch goals isolated	All objectives incl. stretch delivered

Chapter 7

Implementation and results

Each section follows the same discipline: *what* was built, *why*, *how*, and what was *measured*—including measurements that invalidated the initial plan.

7.1 Parsing and its verification

Drain (benchmark configurations) parses the full HDFS log into **48** templates and BGL into **429**. Correctness is enforced by a structural invariant rather than spot checks: the HDFS template for block allocation must occur exactly once per block; its count, 575,061, matches the label file exactly. A parsing error here would silently corrupt every downstream stage, which justifies an exact test.

7.2 Exploratory analysis: three findings that shaped the design

Finding 1 — events cluster massively. The Fano factor (interval-count variance/mean; $= 1$ for Poisson) measures 5.3 on HDFS per-minute counts and 698–2286 on BGL: one to three orders of magnitude above memorylessness. Self-excitation is a first-class property of the data, established *before* any model (Figure 7.1, left).

Finding 2 — HDFS cannot host a continuous-time model. HDFS stamps time to the second at ≈ 80 events/s. Measured: **98.8%** of consecutive events tie globally; **90.5%** tie *within* a block; every block lives ≤ 54 s (median 7 s) over a median of ≈ 2 distinct seconds. A continuous-time process requires an order that does not exist here. Artificial jitter was rejected—the estimated branching ratio would measure the jitter, not the system—and the Hawkes analysis was relocated to BGL (microsecond stamps; 0% ties). This mid-project redirection, forced by measurement, is presented as a result in its own right.

Finding 3 — the base rate drifts. Under the temporal split the anomaly rate falls from 3.26% (training period) to 2.16% (test period): non-stationarity is present even in the labels, anticipating the drift analyses below. The block-size distribution (median 19 events) has a long anomaly-rich tail (Figure 7.1, right).

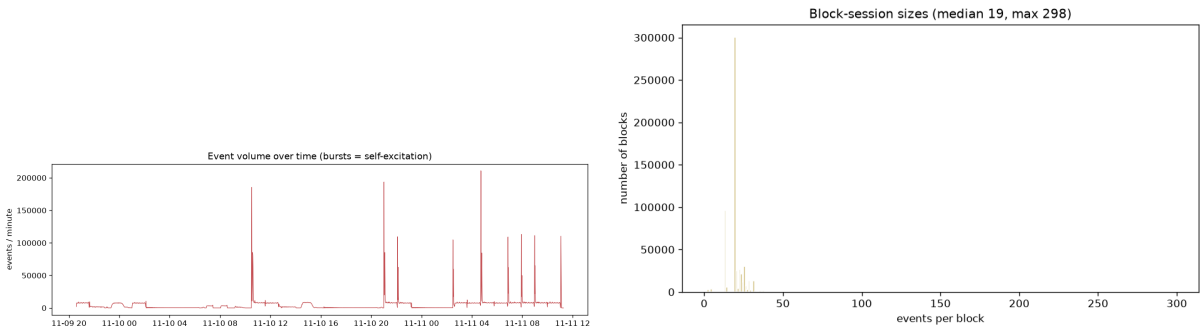

Figure 7.1: Left: HDFS event volume per minute—bursts over a flat baseline. Right: block-session sizes (median 19, maximum 298).

7.3 Detection on HDFS

7.3.1 Set-up

Unit: block session; features and split as specified in Chapter 6. Baseline: Isolation Forest on $\log(1+x)$ counts. Detector: MLP autoencoder $48 \to 32 \to 16 \to 8 \to 16 \to 32 \to 48$ (ReLU), Adam, early stopping on a held-out slice of normal training data (converged in ≈ 19 epochs); score = reconstruction MSE.

7.3.2 Headline results

Table 7.1: HDFS detection, temporal test split (172,518 blocks, 2.16% anomalous).

Metric	Isolation Forest	Autoencoder
PR-AUC	0.748	**0.987**
Precision at recall 0.90	0.556	**0.961**
False alarms / day at recall 0.90	∼12,700	**∼650**

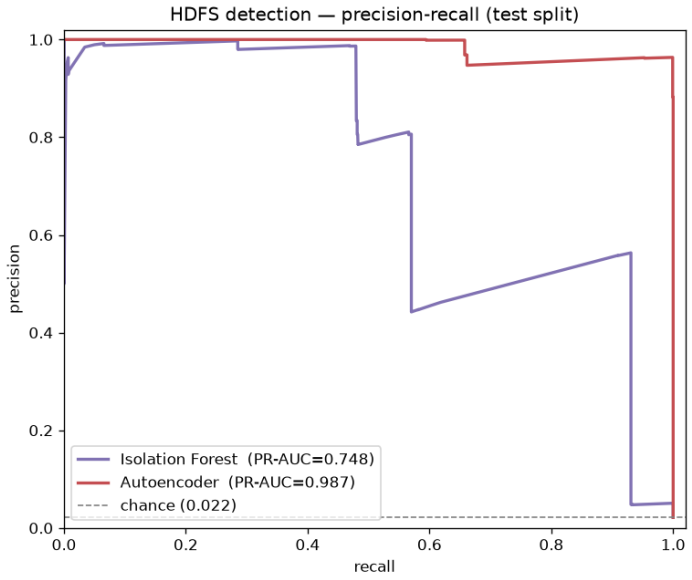

Figure 7.2: Precision–recall on the HDFS test split; chance level 0.022.

At the committed 90% detection rate, the autoencoder divides the false-alarm burden by ≈ 20. Against the 0.022 chance line, PR-AUC 0.987 indicates near-perfect ranking.

7.3.3 Drift decomposition

Table 7.2 re-evaluates both detectors on four consecutive slices of the $\approx$ 5-hour test window, threshold fixed once.

Table 7.2: Drift decomposition (threshold fixed on the full test set).

	slice 1	slice 2	slice 3	slice 4
Anomaly rate	2.90%	3.15%	1.03%	1.57%
IsoForest PR-AUC	0.977	0.692	0.762	0.829
IsoForest FA/day	95	22,019	27,030	1,746
AE PR-AUC	**0.999**	**0.995**	**0.989**	**1.000**
AE FA/day	19	228	2,221	133

Two lessons. The baseline's *ranking itself* is unstable (PR-AUC 0.69–0.98), while the autoencoder's is uniformly excellent (≥ 0.989)—stability an aggregate would have hidden. Yet even for the autoencoder the *false-alarm rate at a fixed threshold* spans two orders of magnitude across slices: ranking is solved, *alerting* is not. This observation drives §7.6.

Positioning

Table 7.1 reproduces a widely published benchmark result (§4.3); it validates the pipeline and supplies realistic scores to later stages. The drift decomposition, the FA/day framing, and what follows are where this project departs from the standard reproduction.

7.4 The Hawkes timing model on BGL

7.4.1 Estimator verification (gate)

Before real data: trajectories simulated by Ogata thinning with known (μ, α, β) are refit—parameters recovered within a few percent; the rescaling diagnostic accepts the true model (KS ≈ 0.01, mean gap ≈ 1) and rejects a deliberately wrong one. Listing 7.1 shows the core recursion exactly as implemented and tested.

```
def _recursion_A(t: np.ndarray, beta: float) -> np.ndarray:
    """A_i = e^{-beta (t_i - t_{i-1})} (1 + A_{i-1}), A_0 = 0."""
    A = np.empty_like(t)
    A[0] = 0.0
    for i in range(1, len(t)):
        A[i] = np.exp(-beta * (t[i] - t[i-1])) * (1.0 + A[i-1])
    return A

def neg_log_likelihood(params, t, T):
    mu, alpha, beta = np.exp(params)          # log-parametrisation
    lam = mu + alpha * _recursion_A(t, beta)  # intensities at events
    integral = mu*T + (alpha/beta) * np.sum(1 - np.exp(-beta*(T - t)))
    return -(np.sum(np.log(lam)) - integral)
```

Listing 7.1: The $O(n)$ likelihood recursion (project source).

7.4.2 Fits on contrasting windows

Table 7.3: Maximum-likelihood Hawkes fits on BGL (24-hour windows, all events).

Window	Events	$\eta = \alpha/\beta$	KS statistic	mean rescaled gap
2005-09-03 (calm)	6,858	**0.971**	0.095	1.0001
2005-06-11 (cascade)	152,669	**0.9998**	0.431	1.0000

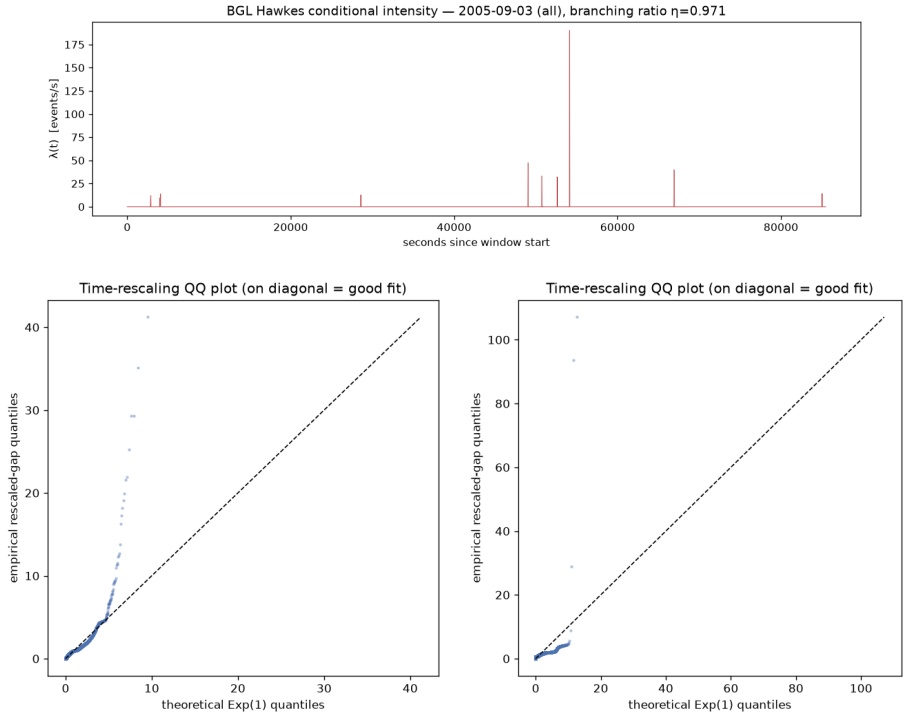

Figure 7.3: Top: fitted $\lambda(t)$, calm window. Bottom: rescaling QQ-plots, calm (left) and cascade (right): diagonal body, departing upper tail.

7.4.3 Interpretation

The branching ratio is *near-critical* on both windows ($\eta \approx 0.97$–1.0): each event triggers on the order of one direct offspring; by the cluster representation, mean cascade size $1/(1-\eta)$ is enormous—the stream operates at the edge of self-sustaining avalanches. This is the project's central quantitative statement about the data, and it is *validated*: the rescaled-gap mean of exactly 1.000 certifies the compensator, and the QQ body on the diagonal certifies the bulk fit. The diagnostics also *localise* the model's failure: the upper tail departs upward—the longest quiet gaps exceed what one exponential timescale allows. After a burst the fitted kernel expects continued activity; when silence follows, the integrated intensity over the gap is inflated. Mild when calm (KS 0.095), severe in the cascade (KS 0.431). Conclusion: the exponential Hawkes captures the clustering to first order, and its own diagnostic argues for a multi-scale (sum-of-exponentials) kernel—future work rather than a defect of method.

7.5 Ablation: does timing help detection?

The timing model must *earn* its place. Same detector (Isolation Forest), same 60-s windows, same temporal split; only the features change.

Table 7.4: Feature ablation on BGL windows (test split, 4.71% anomalous).

Feature set	PR-AUC	Precision at recall 0.90	FA / day
Content	0.081	0.046	114
Timing	0.087	0.083	61
Content + timing	0.090	0.071	72
Chance	0.047	—	—

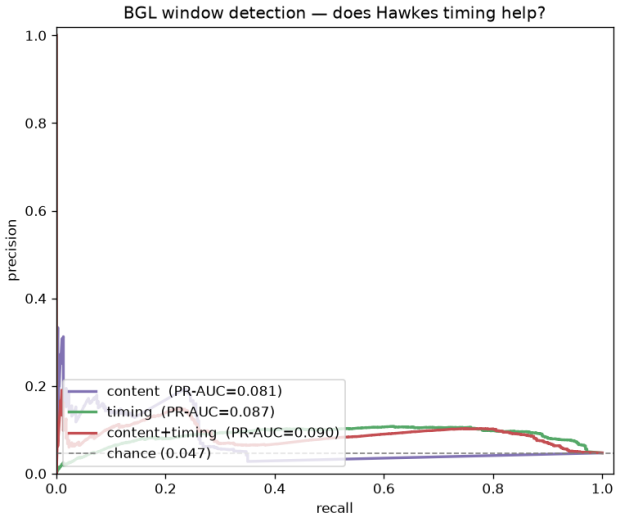

Figure 7.4: Ablation PR curves: content collapses to chance at moderate recall; timing holds $\approx 2\times$ chance across the range.

Both faces are reported. Absolutely, the task is hard: every feature set sits near PR-AUC 0.09 (a 60-s window dilutes a few alert lines in heavy traffic). Relatively, *timing is the strongest signal*: at recall 0.90 it halves the false alarms of content features and nearly doubles precision. The self-excitation level carries operational information that counts do not—a measured, honest, affirmative answer.

7.6 Adaptive thresholding under drift

7.6.1 EVT: verification and a documented failure mode

The POT threshold (5.5) with the method-of-moments GPD estimator was verified on simulation: target exceedance rates held on stationary streams; drift tracked where a fixed threshold fails. On the real HDFS scores, however, extrapolation misbehaved—thresholds could fall below the bulk. Diagnosis: **58.7%** of normal test blocks share one *identical* score value (normal sessions are so stereotyped that whole cohorts have byte-identical count vectors); the score law has large atoms, violating the continuity hypothesis of §5.6. The design response: on HDFS, replace the parametric tail by the *distribution-free* trailing-window quantile; reserve EVT for continuous scores (e.g. the BGL timing features). Reporting this failure mode is deliberate: knowing *when a tool's hypotheses fail* is as valuable as the tool. Listing 7.2 shows the substituted mechanism in full—its simplicity is the point: it assumes nothing about the score distribution.

```
class RollingQuantile:
    """Alert threshold = empirical (1-q)-quantile of a trailing window,
    recomputed every `stride` observations."""
    def __init__(self, q=0.02, window=20_000, stride=500):
```

```
        self.q, self.window, self.stride = q, window, stride
    def fit(self, calib):
        self._buf = list(calib[-self.window:])
        self.z = float(np.quantile(self._buf, 1 - self.q))
        return self
    def run(self, scores):
        flags, thr, since = [], [], 0
        for x in scores:
            flags.append(x > self.z)           # alert decision first
            self._buf.append(float(x))         # then update the window
            del self._buf[:-self.window]
            since += 1
            if since >= self.stride:           # periodic recalibration
                self.z = float(np.quantile(self._buf, 1 - self.q))
                since = 0
            thr.append(self.z)
        return np.array(flags), np.array(thr)
```

Listing 7.2: Distribution-free adaptive threshold (trailing window quantile), as deployed on HDFS.

7.6.2 Drift-stabilisation result

Target flag rate $q = 3\%$; calibration on the first 20,000 test blocks; deployment on the remaining 152,518.

Table 7.5: False-alarm stability under drift (HDFS autoencoder scores, $q = 3\%$; six consecutive slices).

FA/day	s1	s2	s3	s4	s5	s6	recall (period)
Fixed	15,104	153,711	861,686	755,217	445,192	3,664	1.00
Adaptive	3,889	0	7,584	4,242	900	1,543	0.72

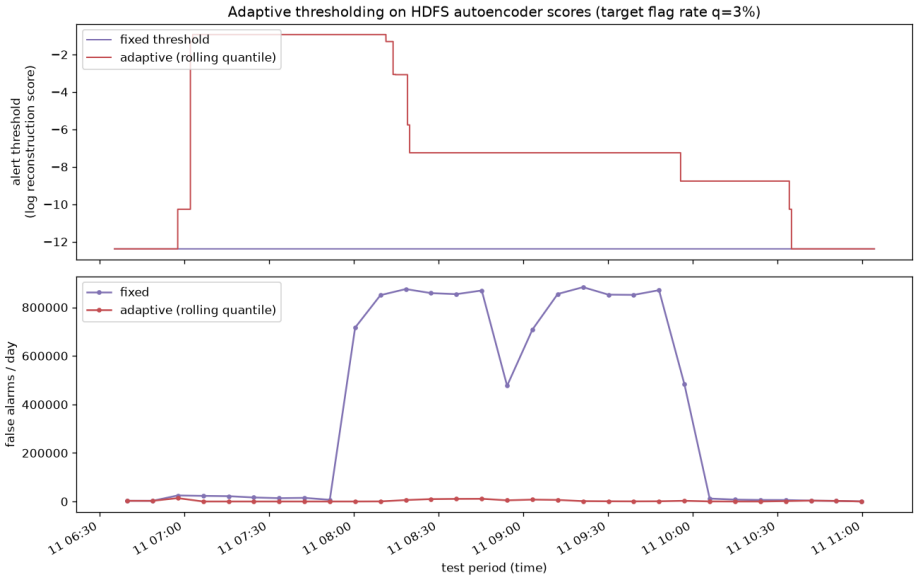

Figure 7.5: Fixed vs. adaptive threshold against the drifting score distribution (top) and resulting false alarms per day (bottom).

The fixed threshold's false alarms explode by a factor $\times 235$ mid-drift (at worst flagging essentially all blocks); the adaptive threshold holds the alert budget throughout. The cost is explicit:

when true anomalies exceed the flag budget, recall drops (1.00 → 0.72 over the period). The trade-off is thereby converted from a hidden failure into a single tunable operating parameter q.

7.7 Triage: from alerts to ranked incidents

The agent applies *deterministic reduction, then explanation*:

1. **Deduplication** — flags with identical signature (entity + template multiset) merge with a count;
2. **Correlation** — temporally clustered flags form one incident: the operational translation of near-critical self-excitation (clustered anomalies are cascade members sharing a root cause);
3. **Explanation and ranking** — each incident is emitted as strict JSON and ranked P1→P4. The backend is pluggable: a deterministic rule engine (always available) or an LLM constrained by the same schema; one call per *incident*, the reduction paying for the model.

On the 200 highest-scoring HDFS test blocks: **90 incidents** (P1:13, P2:53, P3:2, P4:22; categories: data-loss 54, replication-failure 30, network 6). Listing 7.3 shows one output object.

```
{ "id": "INC-0009", "priority": "P1", "category": "replication_failure",
  "summary": "10 correlated anomalies over 105s across 10 entities;
              dominant category 'replication_failure'.",
  "recommended_action": "Check DataNode health and trigger re-replication
                         of under-replicated blocks.",
  "n_anomalies": 10, "dedup_count": 10, "max_score": 1.0,
  "start": "2008-11-11 06:56:48", "end": "2008-11-11 06:58:33" }
```

Listing 7.3: Example triaged incident (rule-based backend).

Honest note

The development environment provided no LLM credential; the demonstration therefore ran on the deterministic backend. The LLM path (schema-constrained structured output) is implemented and unit-tested but was not exercised against a live model in the reported runs.

7.8 Demonstrator and analysis notebook

A Streamlit application presents the pipeline in four views (Detection, Thresholding, Hawkes, Incidents with drill-down); it was verified headlessly (health endpoint and rendered screenshots, Figure 7.6). In addition, a fully-executed Jupyter notebook (`notebooks/full_pipeline.ipynb`) reproduces the entire analysis with narrative markdown—data, EDA, detection, Hawkes fits, ablation, thresholding, triage—so that every figure of this chapter can be inspected and re-run interactively.

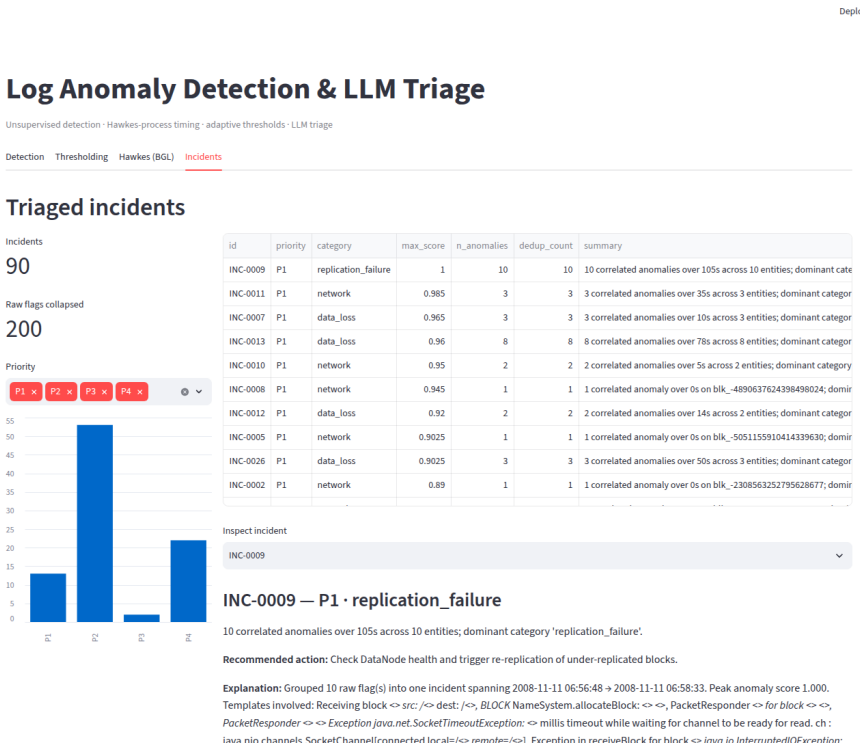

Figure 7.6: Demonstrator, Incidents view: 200 flags reduced to 90 ranked, explained incidents.

Chapter 8

Discussion

8.1 Synthesis

Table 8.1: Consolidated results of the internship.

Item	Result
HDFS parsing	11.2M lines → 48 templates (invariant-verified)
BGL parsing	4.71M lines → 429 templates
Detection (HDFS test)	PR-AUC **0.987** vs. 0.748 baseline
Operational gain	**∼20×** fewer false alarms at recall 0.90
Hawkes branching ratio	$\eta \approx 0.97$–1.0 (near-critical), rescaling-validated
Timing ablation	false alarms **halved** vs. content features
Thresholding	FA/day swing ×235 → **held near budget** under drift
Triage	200 flags → **90 ranked incidents**
Quality	34 automated tests; fully reproducible; executed analysis notebook

8.2 Limitations and threats to validity

1. **Reproduction, not novelty, on HDFS detection.** The 0.987 figure validates the pipeline; the literature contains many equivalent results.
2. **Absolute difficulty of BGL windows.** PR-AUC ≈ 0.09 (vs. 0.047 chance) is weak in absolute terms; the timing gain is relative and is presented as such.
3. **Kernel misfit in the tail.** The exponential kernel is rejected in the upper tail by its own diagnostic; the cascade-window $\eta \approx 0.9998$ must be read with that misfit in mind.
4. **EVT inapplicable to discrete scores.** On HDFS the GPD hypothesis fails (58.7% identical scores); the distribution-free alternative was used. EVT's intended domain here (continuous timing scores) was verified on simulation, not yet in production-like streams.
5. **LLM backend not exercised live** in the reported runs (no credential in the development environment).
6. **Hyperparameters fixed a priori** (window 60 s, top-50 vocabulary, two decay scales, architecture); no sweep was performed, so reported numbers are conservative.
7. **Public data as proxy.** Transfer to an internal log source —formats, volumes, drift regimes— is prepared by the design (generic structured-log interface) but not performed within this internship.
8. **Single-run variance.** Isolation-Forest results carry run-to-run randomness; the ablation's PR-AUC gaps (±0.006) are within that noise, which is why the argument rests on the operating-point (false-alarm) improvement rather than on third-decimal AUC differences.

8.3 Skills developed

On the scientific side: point-process estimation and validation (likelihood, recursion, simulation, rescaling), evaluation methodology for rare events, extreme-value methods and their hypotheses, applied deep learning for unsupervised detection. On the engineering side: packaging, automated testing, reproducibility, incremental delivery under version control. On the professional side: scoping a two-month project, weekly milestone discipline, and—most transferable of all—the habit of reporting negative findings with the same rigour as positive ones.

Chapter 9

Conclusion and perspectives

9.1 Conclusion

This internship delivered a complete, reproducible log-anomaly pipeline whose stages answer, term for term, the structural difficulties of industrial log analysis identified in Part I: unsupervised detection for the absence of labels; a validated self-exciting timing model for the cascade structure of incidents; drift-robust alerting and LLM-assisted triage for the volume asymmetry. Scientifically, the central results are a near-critical, rescaling-validated branching ratio on BGL; a measured halving of false alarms attributable to timing features; and a demonstrated stabilisation of the false-alarm rate under drift. Methodologically, the through-line—synthetic verification before real data, strictly out-of-time evaluation, operationally meaningful headline metrics, drift measured rather than averaged away, and negative findings reported as first-class results—is what makes each number in this report defensible, and is, in the author's view, the internship's most durable outcome.

9.2 Perspectives

1. **Multi-scale kernels.** Sum-of-exponentials or power-law excitation to resolve the upper-tail misfit; model selection by the same rescaling diagnostics.
2. **Rescaling residuals as a detector.** The fitted model turns the stream into calibrated Exp(1) material; departures are continuous anomaly scores—squarely in EVT's domain, closing the loop between Chapters 5 and 7.
3. **Systematic sweeps.** Window length, vocabulary, decay scales, detector architecture, with the ablation harness as the evaluator.
4. **Transfer to an internal source.** Deploy the pipeline on an operational log stream of the host entity, exercise the LLM backend live, and measure the end metric that matters: analyst time saved per incident.

Bibliography

[1] P. He, J. Zhu, Z. Zheng, M. R. Lyu. Drain: An Online Log Parsing Approach with Fixed Depth Tree. *IEEE ICWS*, 2017.

[2] S. He, J. Zhu, P. He, M. R. Lyu. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. *IEEE ISSRE*, 2023.

[3] S. He, J. Zhu, P. He, M. R. Lyu. Experience Report: System Log Analysis for Anomaly Detection. *IEEE ISSRE*, 2016.

[4] M. Du, F. Li, G. Zheng, V. Srikumar. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. *ACM CCS*, 2017.

[5] W. Meng et al. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. *IJCAI*, 2019.

[6] F. T. Liu, K. M. Ting, Z.-H. Zhou. Isolation Forest. *IEEE ICDM*, 2008.

[7] A. G. Hawkes. Spectra of Some Self-Exciting and Mutually Exciting Point Processes. *Biometrika*, 58(1), 1971.

[8] Y. Ogata. On Lewis' Simulation Method for Point Processes. *IEEE Trans. Information Theory*, 27(1), 1981.

[9] E. N. Brown, R. Barbieri, V. Ventura, R. E. Kass, L. M. Frank. The Time-Rescaling Theorem and Its Application to Neural Spike Train Data Analysis. *Neural Computation*, 14(2), 2002.

[10] D. J. Daley, D. Vere-Jones. *An Introduction to the Theory of Point Processes*, Vol. I. Springer, 2003.

[11] J. Pickands III. Statistical Inference Using Extreme Order Statistics. *Annals of Statistics*, 3(1), 1975.

[12] S. Coles. *An Introduction to Statistical Modeling of Extreme Values*. Springer, 2001.

[13] A. Siffer, P.-A. Fouque, A. Termier, C. Largouët. Anomaly Detection in Streams with Extreme Value Theory. *ACM KDD*, 2017.

Appendix A

Notation

Symbol	Meaning
$\lambda(t)$	conditional intensity of a point process
μ, α, β	Hawkes background rate, excitation jump, decay rate
$\eta = \alpha/\beta$	branching ratio (mean direct offspring per event)
$\bar{\lambda} = \mu/(1-\eta)$	stationary mean rate (subcritical regime)
$\Lambda(t)$	compensator $\int_0^t \lambda(s)\, ds$
$\Delta\tau_i$	rescaled increment; Exp(1) under a correct model
(γ, σ)	GPD shape and scale
z_q	threshold exceeded with target probability q
x_u, y_u, $s(u)$	features, latent label, anomaly score of unit u
TP, FP, FN, TN	confusion counts at a given threshold

Appendix B

Glossary

Template / event type
The fixed shape of a family of log lines with variable tokens masked; produced by Drain.

Block session (HDFS)
All lines referencing one HDFS block; the labelled unit of analysis.

Count vector
Representation of a unit by its per-template occurrence counts.

Autoencoder
Bottlenecked network trained to reconstruct its input; reconstruction error serves as anomaly score.

Isolation Forest
Ensemble detector scoring by ease of isolation under random splits.

PR-AUC
Area under the precision–recall curve; chance level equals the positive rate.

False alarms per day
False positives above the threshold achieving a fixed recall, divided by the measured test duration.

Temporal split
Train on the earliest period, test on the strictly later one; never shuffled.

Hawkes process
Self-exciting point process; each event raises the intensity with decaying effect.

Branching ratio
Mean number of direct offspring per event; < 1 stable, ≥ 1 explosive.

Compensator / time-rescaling
Integrated intensity; under a correct model, rescaled gaps are Exp(1)—the basis of the goodness-of-fit test.

EVT / POT / GPD
Extreme-value machinery for principled tail thresholds.

Drift
Temporal change of the data or score distribution; measured per period.

Appendix C

Reproducibility

All code, tests, figures, the analysis notebook, and this report's sources are version-controlled in the project repository. Full reproduction (Python $\geq$ 3.10):

```
pip install -r requirements.txt && pip install --no-deps logparser3 && pip install
-e .
python scripts/download.py --dataset HDFS --full
python scripts/parse.py --dataset HDFS --input data/raw/HDFS.log
python scripts/build_features.py --input data/interim/HDFS.log_parsed.csv --labels
data/raw/anomaly_label.csv
python scripts/run_baseline.py
python scripts/train_autoencoder.py
python scripts/run_evt.py --q 0.03
python scripts/run_triage.py --top 200
python scripts/download.py --dataset BGL --full
python scripts/parse.py --dataset BGL --input data/raw/BGL.log
python scripts/fit_hawkes.py --start 2005-06-11 --hours 24
python scripts/bgl_experiment.py --window 60 --top-k 50
python -m pytest tests/     # 34 automated tests
jupyter notebook notebooks/full_pipeline.ipynb
```

Appendix D

Repository layout

```
src/logtriage/         the installable package (single source of truth)
  config.py          dataset definitions (formats, regexes, URLs)
  data/              download and dataset loading
  parsing/           Drain wrapper, canonical schemas
  eda.py             quality report and standard plots
  features/          block sessions, windows, temporal split
  models/            Isolation Forest baseline, autoencoder
  hawkes/            intensity, likelihood, simulation, rescaling
  eval/              metrics (PR, FA/day, drift), EVT thresholds
  triage/            incidents, explainer backends, agent
scripts/               one runnable entry point per stage
tests/                 34 automated tests
notebooks/full_pipeline.ipynb  executed analysis notebook
app/dashboard.py        Streamlit demonstrator
docs/                  technical notes, figures, this report
```

Appendix E

Automated test suite

Module	Properties verified
`test_metrics`	operating point at fixed recall; FA/day on synthetic cases with known answers; PR-AUC edge cases; error on zero positives
`test_hawkes`	parameter recovery from Ogata-thinning simulation; rescaling accepts the true model and rejects a wrong one; compensator strictly increasing; likelihood sanity
`test_evt`	POT holds target exceedance rate; adaptive threshold tracks drift where fixed fails; rolling quantile stabilises discrete drifting scores; GPD moment estimator on exponential tails
`test_features`	count-matrix construction; temporal ordering of sessions; split correctness and boundary; rejection of invalid fractions
`test_parsing`	block-id extraction; timestamp parsing (HDFS and BGL formats, microseconds); malformed-line handling; data-quality report
`test_triage`	deduplication by signature; correlation by temporal gap; category and priority rules; ranking order; record schema
`test_autoencoder`	flags out-of-distribution inputs; refuses to score before fit